

Day 36–37: Multithreading

- Thread class
- Runnable interface
- Thread lifecycle
- Synchronization

Day 36–37: Java Multithreading

We will learn:

1. What is a Thread?
2. What is Multithreading?
3. Process vs Thread
4. Why do we need Threads?
5. Thread class
6. Creating a Thread using Thread
7. start() vs run()
8. Runnable interface
9. Thread class vs Runnable
10. Thread lifecycle
11. Thread states
12. Thread methods
13. sleep()
14. join()
15. interrupt()
16. Thread scheduling
17. Race condition
18. Synchronization
19. synchronized method
20. synchronized block

- 21. Lock / monitor concept
 - 22. Real-world examples
 - 23. Common mistakes
 - 24. Interview questions
 - 25. Practice programs
-

1. First: What is a Thread?

Before understanding Thread, let's understand **program execution**.

Suppose we have:

```
System.out.println("Hello");
```

```
System.out.println("Java");
```

```
System.out.println("World");
```

Java normally executes these statements **one after another**.

Statement 1

↓

Statement 2

↓

Statement 3

This is called **sequential execution**.

Thread means?

A Thread is a small unit of execution inside a program.

Simple ga:

Thread ante program lo oka task ni execute chese execution path.

For example, imagine a YouTube application.

At the same time:

YouTube Application

|

+---- Play Video

|

```
+---- Download Video
|
+---- Show Comments
|
+---- Load Recommendations
```

These different activities can be handled using different threads.

2. What is Multithreading?

Multithreading means executing multiple threads within the same program.

Simple definition:

Multithreading is a Java feature that allows multiple threads to execute concurrently within a program.

Example:

Application

```
|
+---- Thread 1 → Download file
|
+---- Thread 2 → Play music
|
+---- Thread 3 → Display UI
```

Instead of one task waiting for another task to completely finish, multiple tasks can make progress concurrently.

3. Real-Life Example

Imagine a restaurant.

There is:

Restaurant

```
|
+---- Waiter 1 → Customer A
|
```

+---- Waiter 2 → Customer B

|

+---- Waiter 3 → Customer C

If there is only one waiter:

Customer A

↓

Finish A

↓

Customer B

↓

Finish B

↓

Customer C

This is similar to **single-threaded execution**.

With multiple waiters:

Waiter 1 → Customer A

Waiter 2 → Customer B

Waiter 3 → Customer C

This resembles **multithreading**.

4. Process vs Thread

This is very important.

Process

A **process** is a running program.

For example, when you open:

Chrome

Chrome is a process or collection of processes managed by the operating system.

When you run:

MyJavaProgram

the operating system creates a process for it.

Thread

A thread is an execution unit **inside a process**.

Think:

Process

|

+---- Thread 1

+---- Thread 2

+---- Thread 3

A process can have multiple threads.

Easy analogy

House = Process

People inside house = Threads

House

|

+--- Person 1

+--- Person 2

+--- Person 3

All people are inside the same house, but each can perform different activities.

5. Why do we need Multithreading?

Suppose we have this:

Task 1 → Download file → 10 seconds

Task 2 → Play music → 10 seconds

Without multithreading:

Download

↓

10 seconds

↓

Play music

↓

10 seconds

Total roughly:

20 seconds

With concurrent execution:

Download —————>

Play music —————>

Both can make progress during the same period.

This can improve **responsiveness** and **resource utilization**, depending on the workload.

6. Important Terminology

Before coding, understand these words.

Concurrency

Multiple tasks make progress during overlapping periods.

Example:

Task A → 

Task B → 

Parallelism

Multiple tasks literally execute at the same time on different CPU cores.

Example:

CPU Core 1 → Task A

CPU Core 2 → Task B

So:

Concurrency ≠ always parallel execution.

Java multithreading can support concurrent execution, and on a multicore CPU threads may execute in parallel.

7. How does Java start?

Every Java application starts execution from:

```
public static void main(String[] args)
```

The JVM creates the **main thread** to execute the main() method.

So:

```
public class Demo {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello");  
    }  
}
```

Conceptually:

JVM

```
|  
+---- Main Thread  
    |  
    +---- main()  
        |  
        +---- println()
```

This means your Java program already has a thread even if you never explicitly create one.

8. Creating Threads in Java

There are two fundamental approaches you need to understand:

Approach 1

Extend the Thread class.

```
class MyThread extends Thread {
```

```
}
```

Approach 2

Implement the Runnable interface.

```
class MyTask implements Runnable {
```

```
}
```

We'll study both deeply.

9. Thread Class

Java provides:

```
java.lang.Thread
```

You don't need to import it because java.lang is automatically available.

You can create your own thread by extending Thread.

Example:

```
class MyThread extends Thread {
```

```
    @Override
```

```
    public void run() {
```

```
        System.out.println("Thread is running");
```

```
    }
```

```
}
```

Then:

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        MyThread t = new MyThread();
```

```
        t.start();
```

```
}  
}
```

Output:

Thread is running

10. Understand this code deeply

Let's break:

class MyThread extends Thread

Meaning:

MyThread

↓

inherits

↓

Thread

So MyThread becomes a thread-capable class.

Then:

@Override

public void run()

run() contains the **work that the thread should perform**.

Example:

```
public void run() {
```

```
    System.out.println("Downloading...");
```

```
}
```

So:

MyThread

|

+---- run()

|
+---- Downloading...

11. What is run()?

run() is the method containing the task performed by the thread.

Example:

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        for (int i = 1; i <= 5; i++) {
```

```
            System.out.println(i);
```

```
        }
```

```
    }
```

```
}
```

Here the thread's task is:

Print 1

Print 2

Print 3

Print 4

Print 5

12. Very Important: start() vs run()

This is one of the **most important multithreading interview questions**.

You might think:

```
t.run();
```

will start a new thread.

No.

Normally, you should use:

`t.start();`

start()

When you call:

`t.start();`

Java asks the JVM/underlying runtime to start a new thread of execution, and that thread eventually invokes `run()`.

Conceptually:

main thread

|

| `start()`

↓

new thread

|

↓

`run()`

run()

If you directly call:

`t.run();`

you're simply calling a normal method.

No new thread is started by that call.

main thread

|

| `run()`

↓

`run()` executes on main thread

Remember:

`start()`

↓

starts new thread

↓

run()

while:

run()

↓

ordinary method call

13. Example: start()

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        for (int i = 1; i <= 5; i++) {
```

```
            System.out.println(i);
```

```
        }
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        MyThread t = new MyThread();
```

```
        t.start();
```

```
        System.out.println("Main thread");
```

```
}  
}
```

Possible output:

1

2

Main thread

3

4

5

Or:

Main thread

1

2

3

4

5

Or another valid interleaving.

Why?

Because both threads are executing concurrently.

Main Thread

```
|  
+---- System.out.println("Main thread")
```

MyThread

```
|  
+---- run()  
    |  
    +---- 1  
    +---- 2
```

+---- 3

+---- 4

+---- 5

The exact order is not something you should assume unless you establish synchronization/order explicitly.

14. Why output order changes?

Because the operating system/JVM scheduler decides when runnable threads get CPU time.

For example:

Main Thread MyThread

| |

| |

| +---- 1

+---- Main

| +---- 2

+---- Done +---- 3

Another run could be:

MyThread

|

+---- 1

+---- 2

Main Thread

|

+---- Main

MyThread

|

+---- 3

So multithreaded programs may have **non-deterministic execution order** unless you control the relationship between threads.

15. Creating Multiple Threads

```
class MyThread extends Thread {  
  
    private String name;  
  
    MyThread(String name) {  
        this.name = name;  
    }  
  
    public void run() {  
  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(name + " : " + i);  
        }  
    }  
}
```

Then:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        MyThread t1 = new MyThread("Thread A");  
        MyThread t2 = new MyThread("Thread B");  
  
        t1.start();  
        t2.start();  
    }  
}
```

```
}  
}
```

Possible output:

```
Thread A : 1  
Thread B : 1  
Thread A : 2  
Thread B : 2  
Thread B : 3  
Thread A : 3
```

But another valid output:

```
Thread A : 1  
Thread A : 2  
Thread A : 3  
Thread B : 1  
Thread B : 2  
Thread B : 3
```

Again, don't depend on the exact scheduling order.

16. Runnable Interface

The second major way to create a thread is:

Runnable

Runnable is a functional interface containing:

```
public interface Runnable {  
    void run();  
}
```

You can implement it:

```
class MyTask implements Runnable {
```

```
    @Override
```

```

public void run() {

    System.out.println("Task is running");

}
}

```

But notice something:

MyTask is **not itself a Thread**.

It represents a **task**.

So we create a Thread:

```
MyTask task = new MyTask();
```

```
Thread t = new Thread(task);
```

```
t.start();
```

Flow:

MyTask

|

| implements Runnable

↓

Task

|

↓

Thread

|

↓

start()

|

↓

run()

17. Complete Runnable Example

class MyTask implements Runnable {

 @Override

 public void run() {

 for (int i = 1; i <= 5; i++) {

 System.out.println(i);

 }

 }

}

public class Main {

 public static void main(String[] args) {

 MyTask task = new MyTask();

 Thread t = new Thread(task);

 t.start();

 }

}

18. Why does Runnable exist?

Important question.

Java supports **single inheritance** for classes.

Suppose:

```
class Student extends Person {  
}
```

Now you want the Student to also inherit:

Thread

You cannot do:

```
class Student extends Person, Thread
```

Java doesn't allow multiple class inheritance.

But you can do:

```
class Student extends Person implements Runnable {  
}
```

So Runnable separates:

WHAT TO DO

from:

WHO EXECUTES IT

This is a very important design idea.

19. Thread vs Runnable

Thread approach

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        // task
```

```
    }
```

```
}
```

Here:

MyThread = Thread + Task

Runnable approach

```
class MyTask implements Runnable {
```

```
public void run() {  
    // task  
}  
}
```

Then:

```
Thread t = new Thread(task);
```

Here:

```
MyTask = Task
```

```
Thread = Worker/Execution mechanism
```

20. Real-world Example of Runnable

Imagine a food delivery system.

Task:

Deliver food

You can represent the task:

```
class DeliveryTask implements Runnable {
```

```
    public void run() {  
        System.out.println("Delivering food");  
    }  
}
```

Then:

```
DeliveryTask task = new DeliveryTask();
```

```
Thread deliveryThread = new Thread(task);
```

```
deliveryThread.start();
```

Conceptually:

```
DeliveryTask
```

|
| WHAT?
↓
Deliver food

Thread
|
| WHO executes it?
↓
Worker thread
This separation is useful.

21. Anonymous Runnable

You can also write:

```
Thread t = new Thread(new Runnable() {
```

```
    @Override  
    public void run() {  
        System.out.println("Task running");  
    }  
});
```

```
t.start();
```

Since Runnable has one abstract method, you can also use a lambda:

```
Thread t = new Thread(() -> {  
    System.out.println("Task running");  
});
```

```
t.start();
```

For now, understand the normal Runnable version first.

22. Thread Lifecycle

This is another very important topic.

A thread doesn't simply exist as:

Created → Finished

It goes through different states.

Java officially represents thread states using:

Thread.State

The important states are:

NEW

RUNNABLE

BLOCKED

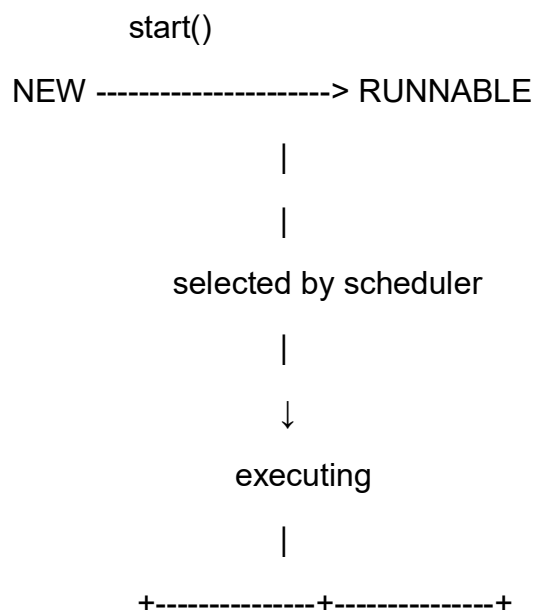
WAITING

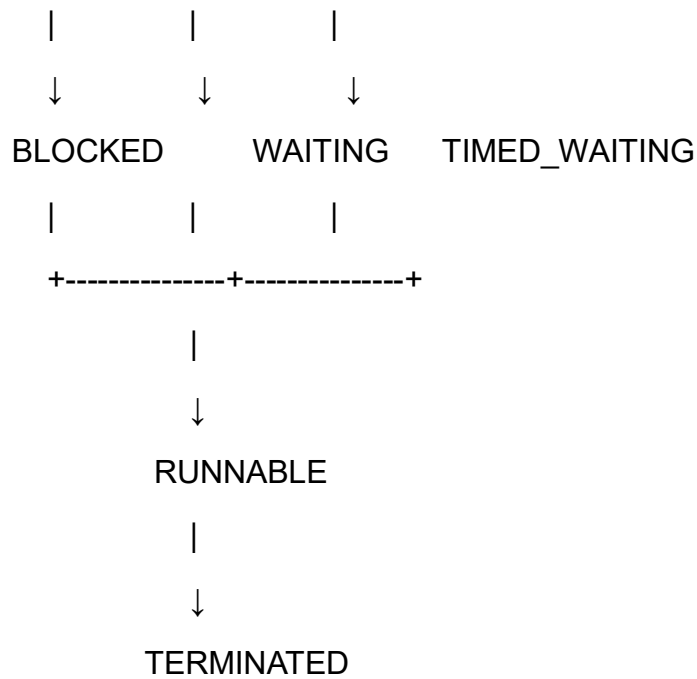
TIMED_WAITING

TERMINATED

23. Thread Lifecycle Diagram

Think:





Important:

Java's RUNNABLE state covers a thread that is **ready to run or actually running**.
 Java's Thread.State enum does not have a separate RUNNING state.

24. State 1: NEW

When you create:

```
Thread t = new Thread();
```

the thread object exists.

But you haven't called:

```
t.start();
```

So:

NEW

Example:

```
Thread t = new Thread();
```

```
System.out.println(t.getState());
```

Output:

NEW

Meaning:

Thread object created, but execution hasn't started.

25. State 2: RUNNABLE

When you call:

`t.start();`

the thread becomes eligible to run.

NEW

↓

`start()`

↓

RUNNABLE

Important:

RUNNABLE does not necessarily mean the CPU is executing it at that exact instant.

It means it is eligible/able to run, subject to scheduling.

26. State 3: BLOCKED

Suppose two threads want the same synchronized lock.

Thread A → has lock

Thread B → wants same lock

Thread B cannot enter that synchronized section until the lock becomes available.

Thread B can be:

BLOCKED

We'll understand this deeply when we learn synchronization.

27. State 4: WAITING

A thread can enter WAITING when it waits indefinitely for another thread to perform some action.

For example:

`thread.join();`

without a timeout can cause the calling thread to wait until the target thread terminates.

Another mechanism is:

`object.wait();`

which waits until another thread performs the appropriate notification.

28. State 5: TIMED_WAITING

This means:

Thread is waiting for a specific amount of time.

Example:

`Thread.sleep(2000);`

The thread sleeps for approximately 2 seconds.

Other examples include timed:

`join()`

`wait()`

29. State 6: TERMINATED

When `run()` finishes:

RUNNABLE

↓

`run()` completed

↓

TERMINATED

Example:

`class MyThread extends Thread {`

`public void run() {`

`System.out.println("Hello");`

```
}  
}
```

After execution completes, the thread is terminated.

A terminated thread cannot be restarted.

30. Can we call start() twice?

No.

Example:

```
Thread t = new Thread() -> {  
    System.out.println("Hello");  
});
```

```
t.start();
```

```
t.start();
```

This causes:

java.lang.IllegalThreadStateException

A thread can be started only once.

31. sleep()

Very important method:

```
Thread.sleep()
```

Example:

```
try {  
    Thread.sleep(2000);  
} catch (InterruptedException e) {  
    System.out.println("Interrupted");  
}
```

2000 milliseconds means:

2000 ms = 2 seconds

During sleep, the thread is in:

TIMED_WAITING

32. Important: sleep does NOT mean CPU is permanently blocked

When you call:

```
Thread.sleep(2000);
```

the current thread pauses for approximately 2 seconds.

It does not keep executing its normal code during that sleep interval.

Also, sleeping does **not** release a monitor lock held by the thread.

This becomes important in synchronization.

33. Why does sleep() throw InterruptedException?

Because another thread can request that a sleeping/waiting thread be interrupted.

So Java forces you to handle:

InterruptedException

Example:

```
try {  
    Thread.sleep(1000);  
} catch (InterruptedException e) {  
    System.out.println("Thread interrupted");  
}
```

34. join()

Suppose:

Main Thread

|

+---- starts Worker

|

+---- continues

But you want main to wait until Worker finishes.

Use:

```
t.join();
```

Example:

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        for (int i = 1; i <= 3; i++) {
```

```
            System.out.println(i);
```

```
        }
```

```
    }
```

```
}
```

Then:

```
public static void main(String[] args) throws InterruptedException {
```

```
    MyThread t = new MyThread();
```

```
    t.start();
```

```
    t.join();
```

```
    System.out.println("Main finished");
```

```
}
```

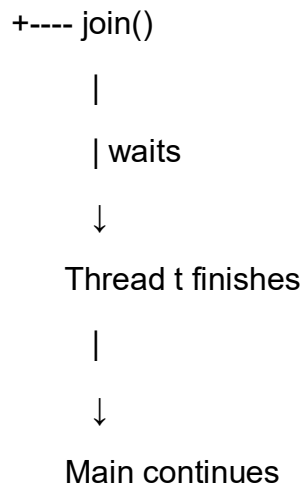
Meaning:

Main

```
|
```

```
+---- start t
```

```
|
```



35. join() Real-World Example

Suppose you need:

Download data



Process data



Display result

Processing should not begin before downloading finishes.

So:

```
downloadThread.start();
```

```
downloadThread.join();
```

```
processData();
```

Here join() establishes the required ordering.

36. getName()

Every thread has a name.

```
Thread t = new Thread();
```

```
System.out.println(t.getName());
```

You can give your own name:

```
Thread t = new Thread();
```

```
t.setName("DownloadThread");
```

```
System.out.println(t.getName());
```

37. currentThread()

This is very important.

```
Thread.currentThread()
```

returns the thread that is currently executing the code.

Example:

```
System.out.println(Thread.currentThread().getName());
```

Inside main() it normally prints:

main

Inside another thread:

```
public void run() {  
    System.out.println(Thread.currentThread().getName());  
}
```

it prints that worker thread's name.

38. Thread Priority

Java threads have priorities.

Constants include:

```
Thread.MIN_PRIORITY
```

```
Thread.NORM_PRIORITY
```

```
Thread.MAX_PRIORITY
```

Typically:

MIN_PRIORITY = 1

NORM_PRIORITY = 5

MAX_PRIORITY = 10

You can write:

```
t.setPriority(8);
```

But don't think:

Priority 10 guarantees that thread executes first.

It does not provide such a guarantee.

Thread scheduling is platform/JVM/OS dependent.

39. Now the Most Important Problem: Shared Data

Multithreading becomes difficult when multiple threads access the **same mutable data**.

Example:

```
int count = 0;
```

Suppose two threads do:

```
count++;
```

You may think:

```
count = count + 1
```

is one operation.

But conceptually it involves multiple steps:

READ count

↓

ADD 1

↓

WRITE count

Suppose:

```
count = 0
```

Two threads:

Thread A

Thread B

Both execute:

count++;

Possible sequence:

A → reads 0

B → reads 0

A → calculates 1

B → calculates 1

A → writes 1

B → writes 1

Final:

count = 1

But you expected:

count = 2

This is called a **race condition**.

40. What is Race Condition?

Simple definition:

A race condition occurs when multiple threads access shared mutable data concurrently and the result depends on the timing/interleaving of their operations.

In Tenglish:

Rendu or more threads same shared data ni same time lo access/change chesthe, execution order batti unexpected result vasthe danini race condition antam.

41. Real-Life Example

Imagine a bank account:

Balance = ₹1000

Two withdrawals happen concurrently:

Thread A → withdraw ₹700

Thread B → withdraw ₹500

If both read:

₹1000

before either writes the updated balance, they may both incorrectly conclude enough money exists.

This is why shared state needs proper coordination.

42. Solution: Synchronization

Java provides:

synchronized

The purpose is to control access to critical sections involving shared state.

Simple definition:

Synchronization ensures that only one thread at a time can execute a particular synchronized critical section for the same lock.

43. What is Critical Section?

A **critical section** is a part of code that accesses shared mutable data and therefore needs coordinated access.

Example:

```
balance = balance - amount;
```

If multiple threads can modify the same balance, this is sensitive code.

So:

Shared Data

↓

Critical Section

↓

Synchronization

44. synchronized Method

Example:

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}
```

Here:

```
synchronized void increment()
```

means that for a given Counter object, only one thread at a time can execute that synchronized instance method.

45. Full Example

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Counter counter = new Counter();
```

```
Thread t1 = new Thread(() -> {
```

```
    for (int i = 0; i < 1000; i++) {
```

```
        counter.increment();
```

```
    }
```

```
});
```

```
Thread t2 = new Thread(() -> {
```

```
    for (int i = 0; i < 1000; i++) {
```

```
        counter.increment();
```

```
    }
```

```
});
```

```
t1.start();
```

```
t2.start();
```

```
t1.join();
```

```
t2.join();
```

```
System.out.println(counter.count);
```

```
}
```

```
}
```

Expected result:

2000

The important part is:

synchronized void increment()

It protects the increment operation from conflicting concurrent access through that object's monitor.

46. What happens internally with synchronized?

This is very important.

Every Java object can be associated with a **monitor** used for synchronization.

When you write:

```
synchronized void increment()
```

for an instance method, the thread must acquire the object's monitor before entering the method.

Conceptually:

Thread A

|

| acquire lock

↓

[Counter Object]

|

↓

increment()

|

↓

release lock

If Thread B comes while Thread A holds that same monitor:

Thread B

|

| wants same lock

↓

BLOCKED

Once Thread A exits the synchronized method:

Thread A

|
↓
release lock

|
↓
Thread B can acquire it

47. Think of synchronized as a Bathroom Key

Imagine one bathroom:

Bathroom

|
+---- One key

Person A takes key:

A → key → bathroom

Person B arrives:

B → key unavailable → waits

A finishes:

A → returns key

Then B:

B → gets key → bathroom

The key is like a **lock/monitor**.

The synchronized block/method is like:

Only one person at a time

48. synchronized Block

You don't always need to synchronize the entire method.

You can synchronize only a particular section.

Example:

```
class Counter {
```

```
int count = 0;
```

```
void increment() {
```

```
    synchronized (this) {
```

```
        count++;
```

```
    }
```

```
}
```

```
}
```

Here:

synchronized (this)

means:

Acquire the monitor associated with this particular object.

49. Why use synchronized block?

Suppose:

```
void method() {
```

```
    // 100 lines of code
```

```
    synchronized (this) {
```

```
        // only 2 lines need protection
```

```
    }
```

```
    // another 100 lines
```

```
}
```

Instead of locking the entire method, you can protect only the critical section.

This can reduce unnecessary locking.

50. Method vs Block

Synchronized method

```
synchronized void increment() {  
    count++;  
}
```

Synchronized block

```
void increment() {  
  
    synchronized (this) {  
        count++;  
    }  
}
```

Both can provide mutual exclusion, but the block gives more precise control over what is protected.

51. What is Mutual Exclusion?

This is a key synchronization concept.

Mutual exclusion means:

At most one thread can enter a particular protected critical section at a time for the same lock.

Tinglish:

Multiple threads

↓

Same critical section

↓

One thread at a time

52. Important: Synchronization is Lock-Based

When a thread enters:

synchronized

it needs the corresponding monitor.

For:

synchronized void method()

the lock is generally:

this object

For:

synchronized (this)

the lock is explicitly:

this

For:

synchronized (SomeClass.class)

the lock is the Class object representing SomeClass.

53. Two Objects = Two Different Locks

Example:

Counter c1 = new Counter();

Counter c2 = new Counter();

Each object has its own monitor.

c1 → Lock 1

c2 → Lock 2

So a thread holding c1's lock does not automatically prevent another thread from entering a synchronized instance method on c2.

This is an important interview concept.

54. Static Synchronized Method

Consider:

static synchronized void method() {

```
}
```

Because the method is static, there is no this object.

The lock is associated with the class object:

MyClass.class

Conceptually:

static synchronized method

↓

MyClass.class

↓

lock

55. Synchronization Does Two Important Things

Synchronization is not only about:

one thread at a time

It also provides **memory visibility / ordering guarantees** around lock acquisition and release.

This matters because each thread can have its own working view of memory, and synchronization establishes happens-before relationships defined by Java's memory model.

For beginner understanding:

Locking helps threads safely coordinate both access to shared state and visibility of updates.

56. volatile vs synchronized

You will encounter volatile later.

Don't confuse them.

volatile

Main purpose:

visibility

It does **not** make compound operations like:

count++;

atomic.

synchronized

Provides:

mutual exclusion

+

visibility/order guarantees

So:

volatile int count;

does not automatically make:

count++;

thread-safe.

57. What is Atomicity?

An operation is **atomic** when it happens as one indivisible operation from the relevant perspective.

Example:

count++;

is not a single indivisible read-modify-write operation.

Conceptually:

read

+

modify

+

write

That's why multiple threads can interfere.

Later you can learn:

AtomicInteger

which provides atomic operations such as:

incrementAndGet()

58. Synchronization and Performance

Synchronization adds coordination overhead.

Suppose:

100 threads

↓

same synchronized block

↓

only one enters at a time

Then much of the work may become serialized.

Therefore:

Synchronize the data that needs protection, but avoid unnecessary locking.

This is one reason synchronized blocks can sometimes be preferable to synchronizing an entire large method.

59. Thread Lifecycle with sleep()

Consider:

```
class MyThread extends Thread {  
  
    public void run() {  
  
        try {  
            Thread.sleep(2000);  
        } catch (InterruptedException e) {  
            System.out.println("Interrupted");  
        }  
    }  
}
```

Lifecycle:

NEW



start()



RUNNABLE



sleep()



TIMED_WAITING



sleep time ends



RUNNABLE



run() ends



TERMINATED

60. Thread Lifecycle with join()

Suppose:

t1.start();

t1.join();

The **thread calling join()** waits for t1 to finish.

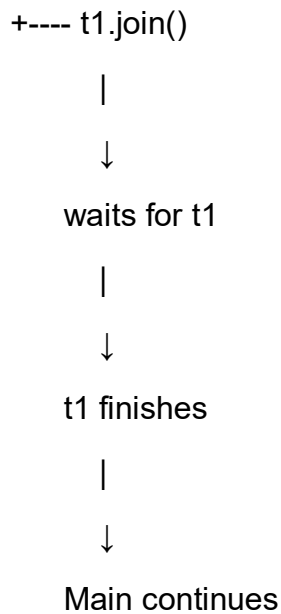
Example:

Main Thread

|

+---- t1.start()

|



61. `sleep()` vs `wait()` vs `join()`

This is a very common interview topic.

Method Purpose

`sleep()` Pause current thread for a duration

`join()` Wait for another thread to terminate

`wait()` Wait for a condition/notification associated with an object's monitor

`sleep()`

```
Thread.sleep(1000);
```

`join()`

```
t.join();
```

`wait()`

```
synchronized (obj) {  
    obj.wait();  
}
```

Don't use `wait()` casually; it belongs to Java's monitor-based coordination mechanism.

62. Important Difference: `sleep` vs `wait`

sleep()

Thread.sleep(1000);

- Method belongs to Thread.
- Causes current thread to enter timed waiting.
- Does **not** release monitors held by the thread.

wait()

obj.wait();

- Method belongs to Object.
- Must be called while owning that object's monitor.
- Releases that object's monitor while waiting.
- Can be resumed through notify() / notifyAll() or interruption.

This distinction is extremely important.

63. notify() and notifyAll()

These are methods of Object.

They are used with wait().

Example:

```
synchronized (obj) {
```

```
    obj.wait();
```

```
}
```

Another thread can do:

```
synchronized (obj) {
```

```
    obj.notify();
```

```
}
```

notify() wakes one waiting thread.

```
notifyAll();
```

signals all threads waiting on that object's monitor.

The awakened threads still need to reacquire the monitor before continuing.

64. Why must wait() be inside synchronized?

Because the thread must own the object's monitor when calling:

wait()

notify()

notifyAll()

Correct:

```
synchronized (obj) {  
    obj.wait();  
}
```

Incorrect:

```
obj.wait();
```

without owning the monitor.

Otherwise:

IllegalMonitorStateException
can occur.

65. Thread Naming

Useful for debugging.

```
Thread t1 = new Thread() -> {  
    System.out.println(  
        Thread.currentThread().getName()  
    );  
});
```

```
t1.setName("DownloadThread");
```

```
t1.start();
```

Output:

DownloadThread

Naming threads is especially useful in larger applications.

66. Daemon Thread

A **daemon thread** is a background-support thread.

Example:

```
Thread t = new Thread() -> {  
    while (true) {  
        System.out.println("Background work");  
    }  
});
```

```
t.setDaemon(true);
```

```
t.start();
```

Daemon status must be set **before** starting the thread.

A JVM can terminate when no live non-daemon threads remain.

Typical examples of background activities in Java applications can use daemon threads, but application design should not assume that daemon work will complete before JVM shutdown.

67. User Thread vs Daemon Thread

Simple idea:

Non-daemon thread



Important application work

Daemon thread



Background/support work

The JVM does not stay alive merely because daemon threads are still running.

68. Thread Scheduler

You may hear:

Thread scheduler decides which thread gets CPU.

Conceptually yes, but the exact scheduling behavior depends on the JVM and operating system.

Suppose:

Thread A → RUNNABLE

Thread B → RUNNABLE

Thread C → RUNNABLE

The scheduler/runtime/OS determines when they execute.

You should **not write code that depends on an assumed scheduling order** unless synchronization establishes that order.

69. Context Switching

Suppose CPU executes:

Thread A

Then switches to:

Thread B

The system has to preserve and restore execution state.

This is called:

Context switching.

Example:

CPU

|

+---- Thread A

|

| context switch

↓

Thread B

|

| context switch

↓

Thread A

Too many threads can create overhead because thread management and context switching are not free.

70. Why Not Create 1 Million Threads?

Because threads consume resources.

Each thread needs runtime resources such as stack space and scheduling/management overhead.

If you create huge numbers of threads carelessly, you can get:

Memory pressure

CPU overhead

Scheduling overhead

Poor performance

That's why real applications commonly use:

Thread Pools

instead of creating unlimited threads manually.

You'll learn this later with:

ExecutorService

71. Thread Pool — Basic Idea

Instead of:

Task 1 → create thread

Task 2 → create thread

Task 3 → create thread

Task 4 → create thread

...

you can have:

Thread Pool

Worker 1

Worker 2

Worker 3

Worker 4

↑

Tasks

Workers are reused for multiple tasks.

This is more scalable for many application workloads.

72. Thread Safety

A class is **thread-safe** when its behavior remains correct when accessed concurrently according to its intended contract.

Example:

```
class Counter {
```

```
    private int count;
```

```
    synchronized void increment() {
```

```
        count++;
```

```
    }
```

```
}
```

The synchronization helps make this particular update operation safe when multiple threads use the same Counter object.

73. Thread-Safe vs Not Thread-Safe

Not protected

```
class Counter {  
  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

Multiple threads can interfere.

Protected

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}
```

Now the increment operation is coordinated through the object's monitor.

74. Full Race Condition Demonstration

```
class Counter {  
  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
}
```

Then:

```
public class Main {

    public static void main(String[] args)
        throws InterruptedException {

        Counter c = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 100000; i++) {
                c.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 100000; i++) {
                c.increment();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(c.count);
    }
}
```

You might expect:

200000

But without proper synchronization, the result can be less because updates can be lost.

75. Fixing the Race Condition

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}
```

Now:

Thread 1

↓

lock

↓

increment

↓

unlock

Thread 2

↓

lock

↓

increment

↓

unlock

The updates are coordinated.

76. Important Concept: Shared Resource

A **shared resource** is data/object that multiple threads can access.

Examples:

Bank balance

Counter

File

Database record

Inventory quantity

Shared collection

Example:

```
Counter counter = new Counter();
```

Both:

Thread A

Thread B

can access:

counter

Therefore it is shared state.

77. Example: Ticket Booking

Suppose:

Available seats = 1

Two users click "Book" at nearly the same time.

Thread A → Book seat

Thread B → Book seat

If both read:

available = 1

before either updates it, both might attempt to reserve the same seat.

The booking operation must be designed to protect the relevant shared state.

This is where synchronization/transactions/other concurrency controls become important depending on the system.

78. Deadlock

Synchronization introduces another important problem:

Deadlock.

Suppose:

Thread A has Lock 1

Thread B has Lock 2

Then:

Thread A wants Lock 2

Thread B wants Lock 1

Now:

Thread A

↓

waiting for Lock 2

↑

|

Thread B

↓

waiting for Lock 1

Neither can continue.

That's a deadlock.

79. Deadlock Example

Conceptually:

```
synchronized (lock1) {
```

```
synchronized (lock2) {  
    // work  
}  
}
```

Another thread:

```
synchronized (lock2) {  
  
    synchronized (lock1) {  
        // work  
    }  
}
```

Possible situation:

Thread A → Lock1 → waiting for Lock2

Thread B → Lock2 → waiting for Lock1

Deadlock.

80. How to Avoid Deadlock?

One common strategy is:

Always acquire multiple locks in a consistent global order.

For example:

Always Lock1 first

Then Lock2

Never:

Thread A → Lock1 → Lock2

Thread B → Lock2 → Lock1

This is an advanced topic, but remember the basic idea.

81. Starvation

Another problem is:

Starvation.

A thread may repeatedly fail to get enough access to required resources because other threads keep getting scheduled or acquiring resources.

Conceptually:

Thread A → keeps getting access

Thread B → keeps waiting

Thread B is experiencing starvation.

82. Livelock

Another concurrency problem:

Livelock.

Threads are not blocked, but they keep reacting to each other and don't make useful progress.

Example conceptually:

Thread A → changes state

Thread B → reacts

Thread A → reacts

Thread B → reacts

...

They're active but not progressing toward completion.

83. Three Problems to Remember

Race Condition



Wrong result due to unsynchronized shared access

Deadlock



Threads wait forever for each other's locks

Starvation



A thread doesn't get enough access to resources

And:

Livelock



Threads keep responding but make no useful progress

84. Thread Class Important Methods

You should remember these:

`start()`

`run()`

`sleep()`

`join()`

`interrupt()`

`getName()`

`setName()`

`getState()`

`currentThread()`

`isAlive()`

`setPriority()`

`getPriority()`

`isDaemon()`

`setDaemon()`

85. `isAlive()`

Checks whether a thread has been started and has not yet terminated.

Example:

```
Thread t = new Thread() -> {  
    System.out.println("Running");  
});
```

```
System.out.println(t.isAlive());
```

```
t.start();
```

```
System.out.println(t.isAlive());
```

The exact second output depends on timing; a very short task might finish before the second check.

So don't assume it will always print true.

86. `getState()`

You can inspect the Java thread state:

```
System.out.println(t.getState());
```

Possible values:

NEW

RUNNABLE

BLOCKED

WAITING

TIMED_WAITING

TERMINATED

87. `interrupt()`

`interrupt()` is a way to request that a thread stop waiting/sleeping or otherwise respond to interruption.

Example:

```
Thread t = new Thread() -> {  
  
    try {  
        Thread.sleep(10000);  
    } catch (InterruptedException e) {  
        System.out.println("Interrupted!");  
    }  
});  
  
t.start();  
  
t.interrupt();
```

The sleeping thread can receive:

InterruptedException

Important:

interrupt() does not forcibly kill a thread.

It is a cooperative interruption mechanism.

88. Don't Confuse interrupt() with stop()

Old thread-stopping approaches such as Thread.stop() are unsafe and deprecated.

Modern code should use cooperative cancellation, often involving:

interrupt()

+

proper handling

89. Runnable vs Thread — Interview Answer

If interviewer asks:

Why is Runnable generally preferred over extending Thread?

You can answer:

Runnable separates the task from the thread that executes it and allows the class to extend another class because Java supports single inheritance for classes. It also fits naturally with thread pools and executor frameworks.

Simple version:

Thread → worker mechanism

Runnable → task

90. One Complete Example

Let's combine:

- Thread
- Runnable
- start
- run
- sleep
- join
- synchronization

```
class Counter {
```

```
    private int count = 0;
```

```
    synchronized void increment() {  
        count++;  
    }
```

```
    int getCount() {  
        return count;  
    }  
}
```

```
class Worker implements Runnable {
```

```
private Counter counter;
```

```
Worker(Counter counter) {  
    this.counter = counter;  
}
```

```
@Override
```

```
public void run() {
```

```
    for (int i = 0; i < 1000; i++) {  
        counter.increment();  
    }
```

```
}  
}
```

```
public class Main {
```

```
    public static void main(String[] args)  
        throws InterruptedException {
```

```
        Counter counter = new Counter();
```

```
        Thread t1 = new Thread(new Worker(counter));
```

```
        Thread t2 = new Thread(new Worker(counter));
```

```
        t1.setName("Worker-1");
```

```
        t2.setName("Worker-2");
```

```

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final count = "
            + counter.getCount());
    }
}

```

Output:

Final count = 2000

91. Trace This Program

Let's understand exactly.

Step 1

```
Counter counter = new Counter();
```

Object created:

Counter Object

count = 0

Step 2

```
new Worker(counter)
```

Two Worker objects/tasks are created, both referring to the **same Counter object**.

Worker 1 —————

↓

Counter

count=0

↑

Worker 2 _____

Step 3

Thread t1 = new Thread(new Worker(counter));

Thread created:

t1

↓

Worker task

↓

same Counter

Step 4

t2 = ...

Second thread also refers to same counter.

Step 5

t1.start();

t2.start();

Now both become eligible to execute.

Thread 1 → run()

Thread 2 → run()

Step 6

Both execute:

counter.increment();

But:

synchronized void increment()

protects the operation.

So:

Thread 1 → acquire lock → increment → release

Thread 2 → acquire lock → increment → release

Or vice versa.

Step 7

Each performs:

1000 increments

Total:

$1000 + 1000 = 2000$

Step 8

`t1.join();`

`t2.join();`

Main waits until both worker threads finish.

Only then:

`System.out.println(...)`

executes.

92. Most Important Mental Model

When learning multithreading, always ask these **five questions**:

Question 1

Which thread is executing this code?

Use:

`Thread.currentThread()`

Question 2

Is this data shared between threads?

Example:

counter

Question 3

Can multiple threads modify it?

If yes, think about thread safety.

Question 4

Is there a critical section?

Example:

count++;

Question 5

How are threads coordinated?

Possibilities include:

synchronized

join()

wait()/notify()

locks

concurrent collections

atomic variables

executors

93. Thread Class vs Runnable — Final Comparison

Feature	Thread	Runnable
Type	Class	Interface
Create by	extends Thread	implements Runnable
Task in	run()	run()
New thread	start()	new Thread(r).start()
Can extend another class?	No	Yes

Feature	Thread	Runnable
Separates task from worker?	Less clearly	Yes
Works naturally with executors?	Less direct	Yes
Common design choice	Useful for simple cases	Often preferred for task definition

94. Thread Lifecycle — Final Table

State	Meaning
NEW	Thread object created but not started
RUNNABLE	Eligible to run / running
BLOCKED	Waiting to acquire a monitor lock
WAITING	Waiting indefinitely for another thread/action
TIMED_WAITING	Waiting for a bounded time
TERMINATED	Execution completed

Remember:

NEW

↓ start()

RUNNABLE

↓

BLOCKED / WAITING / TIMED_WAITING

↓

RUNNABLE

↓

TERMINATED

Not every thread visits every state.

95. Important Interview Questions

1. What is a thread?

A thread is a unit of execution within a process.

2. What is multithreading?

Executing multiple threads concurrently within a program.

3. How can you create a thread in Java?

Common basic approaches:

extends Thread

or:

implements Runnable

Modern applications often use higher-level concurrency APIs such as executors.

4. Difference between start() and run()?

start()

→ starts a new thread of execution

run()

→ contains the task; direct invocation is just a normal method call

5. Can a thread be started twice?

No.

It results in:

IllegalThreadStateException

6. What is a race condition?

When concurrent access to shared mutable state causes the result to depend on timing/interleaving.

7. What is synchronization?

A mechanism for coordinating access to shared state, including mutual exclusion and memory visibility guarantees.

8. What is a synchronized method?

A method whose execution is protected by the monitor associated with its target object for instance methods, or the class object for static methods.

9. What is a synchronized block?

A specific section of code protected using a chosen lock object.

```
synchronized (lock) {  
    // critical section  
}
```

10. Does sleep() release the lock?

No.

This is extremely important.

11. Does wait() release the object's monitor?

Yes.

The thread releases that object's monitor while waiting.

12. What does join() do?

It makes the calling thread wait until the target thread terminates, unless a timed version is used.

13. What is deadlock?

Two or more threads become permanently unable to proceed because each is waiting for a resource held by another.

14. What is thread-safe code?

Code designed so that concurrent use produces correct behavior according to its contract.

96. Common Beginner Mistakes

Mistake 1

```
t.run();
```

thinking it creates a new thread.

Remember:

`start()` → new thread execution

`run()` → task method

Mistake 2

Calling:

```
t.start();
```

```
t.start();
```

Don't do this.

Mistake 3

Thinking:

```
count++;
```

is automatically thread-safe.

It isn't.

Mistake 4

Thinking `sleep()` releases locks.

It doesn't.

Mistake 5

Thinking synchronized means the entire application becomes single-threaded.

No.

Only code protected by the **same lock** is mutually excluded.

Other unrelated code can still execute concurrently.

Mistake 6

Thinking thread priority guarantees execution order.

It doesn't.

Mistake 7

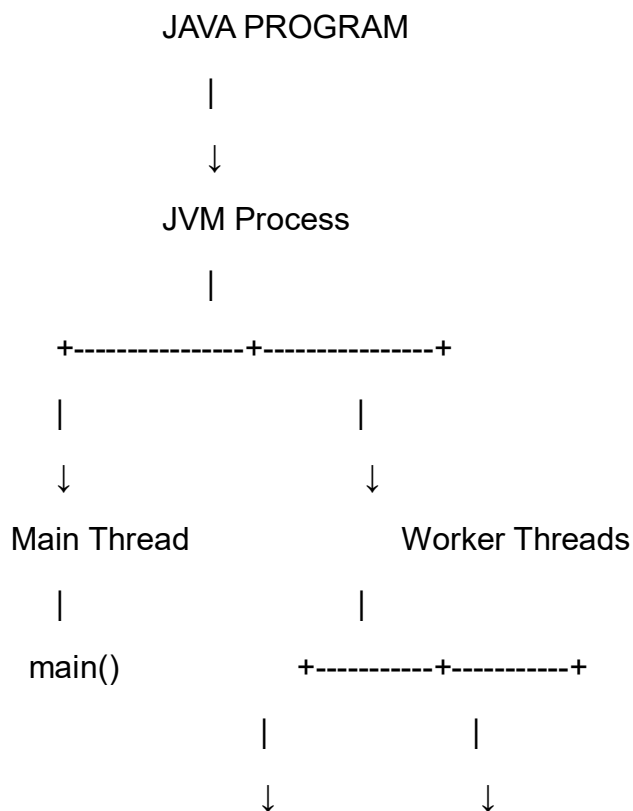
Thinking interrupt() kills the thread.

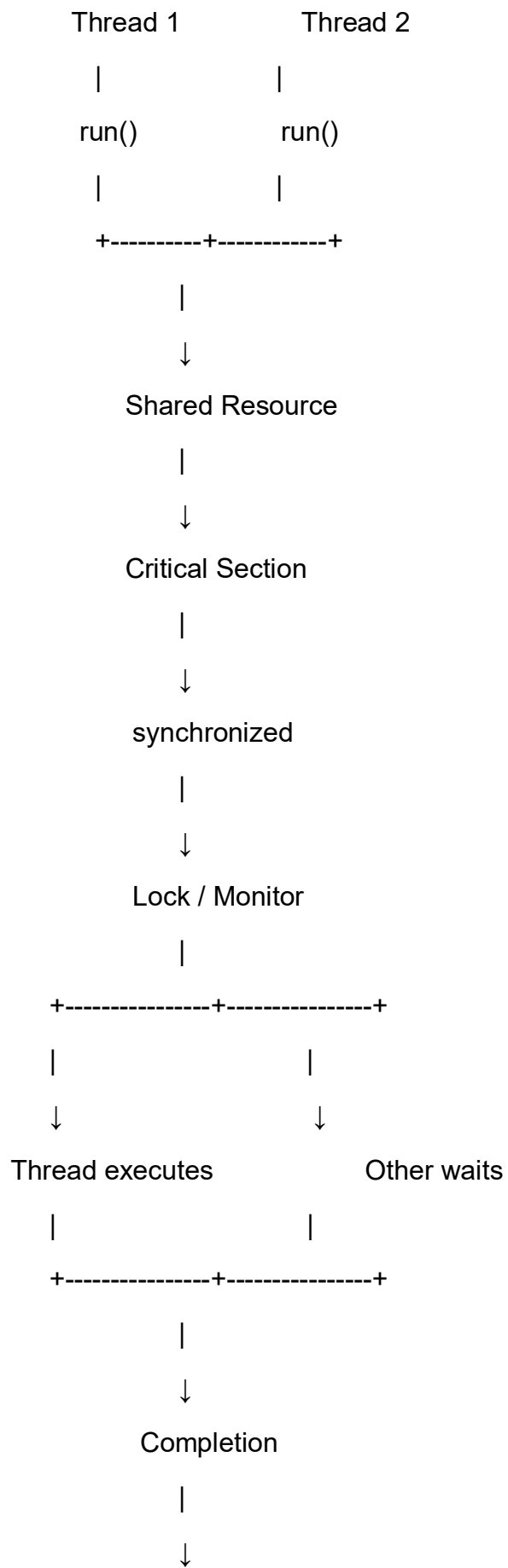
It doesn't.

It requests interruption/cooperative response.

97. One Big Picture

Keep this diagram in your mind:





TERMINATED

98. What You Should Remember From Day 36–37

If you remember only the following, you have the foundation:

1. Thread

↓

Unit of execution

2. Multithreading

↓

Multiple threads making progress concurrently

3. Thread

↓

extends Thread

4. Runnable

↓

implements Runnable

5. start()

↓

starts thread execution

6. run()

↓

contains the task

7. sleep()

↓

pauses current thread for a duration

↓

does NOT release monitor locks

8. join()

↓

wait for another thread to finish

9. Race condition

↓

unsafe concurrent access to shared mutable state

10. synchronized

↓

mutual exclusion + memory visibility/order guarantees

11. synchronized block

↓

protect specific critical section

12. Lock / Monitor

↓

controls synchronized access

13. Thread states

↓

NEW

RUNNABLE

BLOCKED

WAITING

TIMED_WAITING

TERMINATED

14. Deadlock

↓

threads waiting forever for each other

15. Runnable

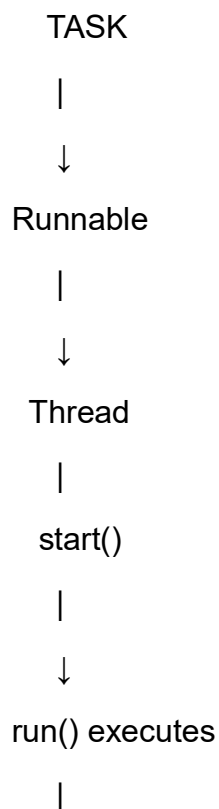
↓

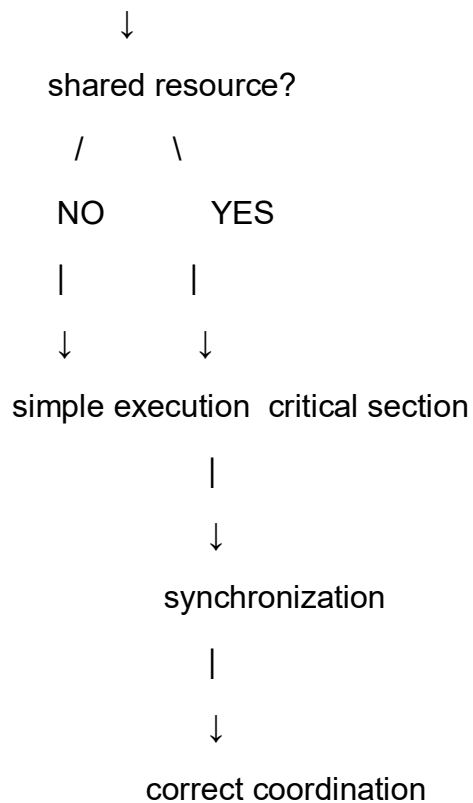
separates task from thread

99. The Most Important Conceptual Difference

Don't memorize multithreading only as syntax.

Understand this:





Once this flow becomes clear, the rest of Java multithreading becomes much easier.

Day 36–37 Practice Set

Try these in this order:

Beginner

1. Create a thread using Thread.
2. Print numbers 1–10 from a thread.
3. Create two threads and print their names.
4. Create a thread using Runnable.
5. Compare start() and run().
6. Use sleep() to print numbers with a delay.
7. Use join() so the main thread waits for a worker.
8. Print the current thread name.
9. Print the thread state before and after start().
10. Create two threads that perform different tasks.

Intermediate

11. Create a shared counter with two threads.
12. Observe the race condition.
13. Fix the counter using synchronized.
14. Rewrite the synchronized method using a synchronized block.
15. Create a bank account shared by two threads.
16. Create a ticket-booking example.
17. Experiment with join().
18. Experiment with interrupt().
19. Create a deadlock example just for understanding.
20. Create a program using multiple Runnable tasks.

Interview-level thinking

For every multithreading problem, ask:

Who is executing?

↓

What data is shared?

↓

Can multiple threads modify it?

↓

What is the critical section?

↓

What synchronization/coordination is needed?

↓

Could race condition/deadlock/starvation occur?

That is the **core thinking pattern** you need for Java multithreading.